

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 13 · LECTURE NOTES

Where It Works

The honest scope, and what to do with it.

MODULE OUTCOMES

- State where reinforcement learning has demonstrably worked.
- Explain what those successes had in common.
- Explain why the same approach fails elsewhere.
- Choose between this course and CSCE 625 for a given problem.
- State what a reinforcement learning result can honestly promise.

1 Where it demonstrably works

Domain	What was achieved	The enabling condition
Board games	Superhuman in Go, chess, and shogi from self-play alone.	A perfect, fast simulator; an unambiguous reward; and self-play's automatic curriculum (Module 11 §2).
Video games	Superhuman in several, including StarCraft II and Dota 2.	A fast simulator and enormous compute — tens of thousands of simulated years (Module 11's closing callout).
Simulated robotics	Locomotion, dexterous manipulation, and some real transfer.	Cheap samples in simulation, plus domain randomisation for transfer (Module 07 §4).
Chip placement, datacentre cooling, compiler tuning	Real deployed improvements over hand-tuned heuristics.	A fast evaluator and a clear scalar objective — and these are the quietest and most convincing successes.
Language model alignment	RLHF — by far the largest deployment of anything in this course.	Cheap rollouts (text generation) and a learned reward model that can be continually corrected (Module 10 §4).
Recommendation and advertising	Real value, and mostly from bandits rather than full reinforcement learning, honestly.	Huge traffic and a short effective horizon — which is Module 02 §1's point about not using an MDP where a bandit suffices.

WHAT THE SUCCESSES HAVE IN COMMON

A cheap, fast simulator or evaluator. Module 01 §2's binding constraint, satisfied — and it is satisfied in every single row of the table, which is the most useful observation in this module.

A reward that is genuinely what you want (a win, a measured latency, a placement score) **or a reward model you can keep correcting as the policy exploits it** (Module 10 §4's continued collection).

A problem where no good hand-written policy existed — so **the comparison against CSCE 625's methods was lost honestly** rather than never run. Note that chip placement and cooling had decades of hand-tuned heuristics, and the learned methods had to beat them.

And a tolerance for occasional failure, since there are no guarantees (Module 01 §1's table). **Check all four against your problem before committing, and note that three of four is not enough** — a problem with a perfect simulator, an unspecifiable reward, and a safety requirement is not a reinforcement learning problem however good the simulator is.

2 Where it fails, and why

- **Real-world interaction is the binding cost.** Physical robots learning on hardware, clinical decision-making, anything with a per-step monetary or safety price. **Module 01 §2's sample counts make it infeasible**, and the responses are Module 08's model-based methods, Module 09's offline methods, or simulation transfer — all of which have their own requirements.
- **The reward cannot be specified. Module 10's problem, and it is not a tooling gap** — a learned reward model helps and is itself gameable, so the difficulty is relocated rather than removed.
- **A guarantee is required.** Safety-critical control, certification, anything with a regulator. **CSCE 625's methods give guarantees and this course's do not** (Module 01 §1's table), and no amount of empirical validation substitutes for a proof when a proof is what is required.
- **A hand-written policy already works well. Then the comparison is lost before it starts** — and **this situation is considerably more common than the literature suggests**, because papers do not get written about problems where a scripted controller was adequate.
- **And the decisions are actually independent of each other. That is supervised learning or a contextual bandit, both far easier and better understood** (Module 02 §1) — and a surprising amount of work framed as reinforcement learning falls here.
- **Each failure is traceable to a specific missing condition from §1, which means the assessment is a checklist rather than a judgement** — and that is the practical form the honesty takes.

3 This course or CSCE 625

	CSCE 625 (classical)	This course (learned)
Requires	A model of the dynamics.	Interaction, in enormous quantity.
Provides	Optimality, proofs, and microsecond runtime (CSCE 625 Module 01 §4).	A policy, and no guarantees of any kind.
Debugging	Trace the search — costs, heuristic values, the frontier.	Seeds, learning curves, and patience (Module 12).
Handles	Known, crisp dynamics.	Unknown dynamics, or dynamics nobody can write down.
Designer control	Direct — edit the heuristic, the tree, the costs.	Indirect, through the reward — which Module 10 establishes is an imprecise instrument.
Try it first?	Yes.	When the first column has honestly failed.

'Try CSCE 625 first' is the recommendation, and it is not a hedge. Most sequential decision problems that look like reinforcement learning problems are planning problems with an unmeasured model — the dynamics are known or knowable, and nobody wrote them down. **Writing them down and planning is frequently a week's work against months of training**, and it produces something debuggable.

AND THE HYBRID IS USUALLY RIGHT

Learn what cannot be written and compute what can. Learn a dynamics model from interaction, then plan with it (Module 08) — which is the most reliable combination and the one with the best sample efficiency.

Or learn a value function or heuristic and use it inside a classical search — which is AlphaZero, and is CSCE 625 Module 05 §3's synthesis: **MCTS with the part nobody could write supplied by learning**.

Or learn the perception and plan over its output, which is CSCE 753 Module 13 §2's division and is how every self-driving stack is built.

In every case the learned component supplies what nobody could write down, and the structured component supplies the guarantees, the speed, and the control. Which is the same conclusion all three courses this semester reached independently — CSCE 753 Module 13, CSCE 625 Module 13, and this one — **and three independent arrivals at one conclusion is the strongest form of evidence this program produces.**

4 Claiming honestly

- 'Mean return 312, interquartile mean 298, across ten seeds, after 5M environment steps; **four of the ten seeds failed to learn the task at all.**' The seeds, the budget, and the failure rate — and the failure rate is the part that is almost always omitted and almost always matters.

- **'It beats a tuned scripted baseline by 18% on this task, and loses to that baseline on two of the five other tasks we tried.'** The baseline tuned, and the losses reported (Module 12 §3).
- **'Trained in simulation; real-system performance is 64% of simulated, measured over 40 physical trials.'** The reality gap quantified (Module 07 §4), which is the only number that matters for a physical system.
- **'The reward is a proxy for X; the agent exploits it by doing Y, which we mitigated by Z and did not eliminate.'** The specification gaming named rather than hidden (Module 10) — and **'did not eliminate' is the honest ending**, because it rarely is eliminated.
- **And what you cannot honestly say: 'the agent learned to play the game'** — unqualified, without the seed count, the sample budget, or the baseline. **Every honest claim here names all three, because Module 12 established that a claim without them is not assessable by anyone**, including by you.

WHERE THIS COURSE, AND THIS SEMESTER, LEAVE YOU

You can implement and diagnose value-based and policy-gradient methods, choose between them from the problem's properties, and recognise which leg of the deadly triad a given method reintroduces — which is the diagnostic skill that makes the algorithm zoo navigable rather than memorised.

You can design a reward and anticipate how it will be gamed, which is **the skill in this course with the most consequence** and the one that transfers furthest beyond it.

And you can evaluate a result to a standard the field frequently does not meet — five seeds, spread shown, baseline tuned equally, protocol and budget stated — which **also means you can read the literature and tell which claims are supported**.

Semester 7 was perception, reasoning, and learning to act. CSCE 753 recovered the world from images; CSCE 625 decided what to do given a known model; and this course learned what to do without one. And the closing rule has not changed across twenty-one courses: state what you measured, state what you assumed, and never claim more than you established. In reinforcement learning it means naming the seeds — because without them, Module 12 says, you have not measured anything.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Irpan — Deep Reinforcement Learning Doesn't Work Yet (free)	The honest assessment that this module's §2 follows , by a practitioner. Dated in its examples and not in its reasoning.
Dulac-Arnold et al. — Challenges of Real-World Reinforcement Learning (free)	§2's failure conditions, enumerated systematically , with the gap between benchmark and deployment settings made explicit.

Resource	What it covers for this module
Mirhoseini et al. — A graph placement methodology for fast chip design (free)	§1's chip placement row — and worth reading alongside its published critiques, as an exercise in Module 12 §4's questions.
Sutton & Barto — chapters 16–17 (free PDF)	Case studies and frontiers , and a reasonable place to choose what to study next.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Check your project's problem against §1's four conditions** and report how many hold.
- 2 **For any that do not, identify which failure in §2 applies.**
- 3 **Build the CSCE 625 solution to your problem** — scripted, planned, or searched — and measure it.
- 4 **Compare your trained agent against it** honestly, across five seeds.
- 5 **Estimate what a model-based approach would cost** in samples, from Module 08's figures.
- 6 **Design a hybrid** for your problem and state which part is learned and why.
- 7 **Take one result from §1's table** and find its sample and compute budget. Report it.
- 8 **Write your project's claim** in the form of §4's list.
- 9 **Revisit what you wrote in Module 01's last exercise** and report what changed.
- 10 **Project 2 is now due.** Submit the agent, the state/action/reward design, the non-learned baseline, five-seed results with spread, the reward-gaming behaviour you found, the shifted-environment evaluation, and a written answer to whether reinforcement learning was the right tool.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six domains where reinforcement learning works and the enabling condition of each.
- 2 What do all of them have in common?
- 3 State the four conditions, and why three of four is insufficient.
- 4 Name five failure conditions and trace each to a missing condition.
- 5 Compare CSCE 625 and this course on six axes.
- 6 Why is 'try CSCE 625 first' not a hedge?
- 7 Give three hybrid architectures and the division in each.
- 8 Give four honest claims and the thing you cannot say.

9 What does 'state what you assumed' mean in this course specifically?